AGENTIC AI ENGINEERING

The Complete Agentic AI Engineer Interview Guide

A structured, in-depth preparation resource covering agent architectures, planning & reasoning strategies, tool use, memory, multi-agent orchestration, agentic RAG, safety & guardrails, evaluation, frameworks, and deployment.

10	53	10	6
CORE TOPICS	Q&A PAIRS	SCENARIOS	CODING EXERCISES

Prepared as a comprehensive interview-readiness reference · Foundational to Advanced coverage · 2026 Edition

TABLE OF CONTENTS

What's inside this guide

01 **Agent Fundamentals & Architectures** — 6 Questions
The opening round for any agentic AI role checks whether you have a precise mental model of what an agent actually is — not the marketing definition, but the architectural one.

02 **Planning & Reasoning Strategies** — 6 Questions
How an agent decides what to do next is the core intellectual problem in agentic AI. Interviewers use this section to see whether you can match a planning strategy to a task's actual structure, not just name-drop ReAct.

03 **Tool Use & Function Calling** — 6 Questions
Tools are how an agent touches the world. Interviewers probe this section hard because tool design mistakes are the single most common source of unreliable agents in production.

04 **Agent Memory Systems** — 6 Questions
Memory design is where a lot of agent projects quietly fall apart in production — context grows unbounded, relevant facts get lost, and costs balloon. This is a favorite deep-dive area for senior-level interviews.

05 **Multi-Agent Systems & Orchestration** — 6 Questions
Once a system involves more than one agent, coordination becomes the central engineering problem. This section tests whether you understand the real coordination patterns, not just that 'multiple agents talk to each other.'

06 **Agentic RAG & Retrieval Strategies** — 5 Questions
Agentic RAG is where retrieval and agency intersect — instead of a fixed retrieve-then-generate pipeline, the agent itself decides what, when, and how to retrieve. This is a frequent deep-dive for roles combining agents with knowledge-intensive tasks.

07 **Safety, Guardrails & Permissions for Agents** — 5 Questions
Agents that take real actions carry real risk. This is consistently the highest-scrutiny section in agentic AI interviews — a candidate who can't speak concretely to agent safety is a hard pass for most teams building production agents.

08 **Evaluating Agentic Systems** — 5 Questions
Evaluating an agent is a fundamentally harder problem than evaluating a single LLM response, because success depends on a whole trajectory of decisions, not one output. This is where senior candidates are most clearly differentiated.

09 **Agent Frameworks & Tooling** — 4 Questions
Interviewers use framework questions to check whether you understand the underlying patterns frameworks implement, not just which library you've used — you should be able to say what a framework buys you and what it costs.

10 **Deployment, Cost & Observability for Agents** — 4 Questions
An agent that works in a demo and an agent that's reliable and affordable in production are very different engineering problems. This section is where 'have you actually shipped this' gets tested.

11 **Coding & Practical Exercises** — 6 Exercises
Hands-on exercises covering agent control-flow building blocks — ReAct loops, supervisor routing, idempotent tool calls, context management, and approval gates.

12 **Scenario-Based Questions** — 10 Scenarios
Real agent fire drills — hand-off loops, fabricated citations, prompt injection via tool results, runaway sandboxes, and autonomy-expansion decisions.

13 **System Design Case Studies** — 3 Cases
Three end-to-end agentic architecture walkthroughs, with the tradeoffs interviewers expect you to raise unprompted.

14 **Project & Portfolio Questions** — 6 Questions
How to structure answers about your own agentic projects so they read as production experience, not tutorials.

15 **Behavioral & Judgment Questions** — 6 Questions
The non-technical questions that decide close calls — autonomy tradeoffs, safety pushback, and cross-functional communication.

16

General Behavioral & HR Questions

6 Questions

Standard cross-role behavioral and HR-round questions — conflict, weaknesses, motivation, and career trajectory.

+

Quick-Reference Glossary

15 Terms

A one-page cheat sheet of every key term and pattern — ideal for a final skim right before the interview.

How to use this guide: Sections 1–10 are tagged Foundational, Intermediate, or Advanced per question. If you're short on time, prioritize Section 5 (Multi-Agent Orchestration), Section 7 (Safety & Guardrails), and Section 12 (Scenarios) — these are where agentic AI interviews most consistently differentiate candidates in 2026.

01

Agent Fundamentals & Architectures

The opening round for any agentic AI role checks whether you have a precise mental model of what an agent actually is — not the marketing definition, but the architectural one.

Q. What precisely makes a system an 'agent' rather than a chatbot or a fixed pipeline?

FOUNDATIONAL

An agent uses an LLM to decide its own sequence of actions at runtime — which tools to call, in what order, whether to continue or stop — based on intermediate results, rather than following a hardcoded sequence a developer wrote in advance. The control flow itself is delegated to the model, not just the content of a single response.

A chatbot (even a sophisticated one) maps an input to a single response. A fixed pipeline (like standard RAG) always executes the same steps in the same order regardless of what it finds along the way. An agent, by contrast, can loop, branch, retry, and change its plan mid-task based on what it observes — that adaptive, model-driven control flow is the defining property, not simply 'it uses tools' or 'it uses an LLM.'

Q. Describe the canonical agent loop: observe, reason, act, repeat. What happens at each stage?

FOUNDATIONAL

Observe: the agent receives the current state — the user's request, prior conversation, and results from any previous actions (tool outputs, retrieved documents, error messages).

Reason: the LLM processes that state and decides what to do next — this can be an explicit reasoning trace (as in ReAct) or an implicit decision embedded directly in a structured tool call.

Act: the agent either calls a tool/function, produces a final answer, or asks a clarifying question — critically, the LLM only proposes the action; the surrounding application code is what actually executes it.

Repeat: the result of the action becomes part of the next observation, and the loop continues until a stopping condition is met — a final answer is produced, a max-step limit is hit, or the task is explicitly abandoned. Every production agent needs an explicit, hard-coded stopping condition; relying on the model to 'decide to stop' is not sufficient on its own.

Q. What is the difference between a single-agent and a multi-agent architecture, and when does multi-agent actually help?

INTERMEDIATE

A single agent handles the entire task with one model, one context, and one set of tools — simpler to build, reason about, and debug, and sufficient for the large majority of real-world use cases.

Multi-agent architectures split a task across multiple specialized agents (e.g., a researcher agent, a coder agent, a reviewer agent) that communicate and hand off work — this helps when different sub-tasks genuinely benefit from different tools, prompts, or context windows that would otherwise conflict or bloat a single agent's context, or when you want isolation (one agent's failure or hallucination shouldn't corrupt another's state).

The honest interview answer: multi-agent systems trade simplicity for specialization, and that trade is often not worth it — a single well-designed agent with good tools frequently outperforms a poorly-decomposed multi-agent system, because multi-agent coordination adds real failure modes (miscommunication, redundant work, inconsistent shared state) that a single agent doesn't have. Reach for multi-agent only when task decomposition provides a clear, demonstrable benefit, not by default.

Q. What is the difference between a stateless (single-turn) agent and a stateful, long-running agent?

INTERMEDIATE

A stateless agent completes its full loop within a single request/response cycle — it receives a task, runs its reasoning/action loop to completion, and returns a result, with no persisted state between invocations beyond what's passed back in.

A stateful, long-running agent persists its state (conversation history, intermediate results, plan, memory) across multiple interactions or even across process restarts — this is necessary for tasks that span minutes/hours/days, involve waiting on external events (a human approval, a long-running job), or need to resume after a failure without starting over.

Building long-running agents introduces real infrastructure requirements that stateless agents don't have: durable state storage, checkpointing so a crash doesn't lose progress, and a way to resume execution from the last checkpoint rather than the beginning — this is a meaningfully different engineering problem than a request-scoped agent loop, and interviewers care whether you recognize that distinction.

Q. What's the difference between an agent framework (LangGraph, AutoGen, CrewAI) and building an agent loop yourself?

INTERMEDIATE

A hand-rolled agent loop gives full control and transparency — you write the observe/reason/act loop explicitly, which makes debugging and customization straightforward but means you're responsible for building state management, retries, and orchestration logic yourself from scratch.

Frameworks provide reusable abstractions for common agentic patterns — state graphs (LangGraph), conversational multi-agent patterns (AutoGen), role-based crews (CrewAI) — plus built-in state persistence, tracing integration, and pre-built patterns for common structures like supervisor/worker hierarchies.

The tradeoff is abstraction cost: frameworks can make simple things faster to build but can obscure exactly what's happening in the loop, which matters a lot when debugging a subtle failure. A strong interview answer shows you've made this tradeoff deliberately — e.g., 'I hand-roll simple, well-defined agent loops for production reliability and predictability, and reach for a framework when I need its state-graph/persistence machinery for a genuinely complex multi-step workflow' — rather than treating framework choice as an arbitrary preference.

Q. What is the difference between agentic and non-agentic use of an LLM with tools (e.g., a single function-calling response vs. an agent)?

FOUNDATIONAL

A single function-calling response is: the model receives a request, decides to call one tool, the application executes it, and the result is returned to the user (possibly after one more LLM call to phrase the answer) — there's no loop, no multi-step planning, and no model-driven decision about whether further action is needed.

An agentic system wraps that same function-calling capability in a loop where the model can decide to call another tool based on what the first one returned, potentially many times, adjusting its plan as it goes — the distinguishing feature is that the number and sequence of steps isn't fixed in advance and is determined by the model's own reasoning about whether the task is complete.

This distinction matters in interviews because many 'agent' projects described by candidates are actually just single-shot function calling — being precise about which one you actually built (and why) signals real understanding rather than buzzword use.

02

Planning & Reasoning Strategies

How an agent decides what to do next is the core intellectual problem in agentic AI. Interviewers use this section to see whether you can match a planning strategy to a task's actual structure, not just name-drop ReAct.

Q. Explain ReAct in detail, and name a concrete failure mode it has.

FOUNDATIONAL

ReAct (Reason + Act) interleaves explicit reasoning traces with actions: the model writes out its reasoning about what to do, takes an action (tool call), observes the result, and reasons again — repeating until it has enough information to answer. The explicit reasoning trace, kept in context, lets each subsequent decision be grounded in the full history of what's been tried and observed so far.

A concrete failure mode: ReAct is fundamentally greedy and sequential — it commits to one action at a time based on current reasoning, with no lookahead or backtracking. If an early step sends it down an unproductive path, it can spend many steps compounding that mistake before recognizing the dead end, since there's no built-in mechanism to reconsider an earlier decision except by continuing to reason forward from where it currently is.

Q. What is Plan-and-Execute, and how does it differ from ReAct?

INTERMEDIATE

Plan-and-Execute separates planning from execution into two distinct phases: first, the model generates a full multi-step plan upfront based on the task (without yet executing anything), then a separate execution loop works through that plan step by step, optionally replanning if a step fails or new information changes what's needed.

ReAct interleaves reasoning and action at every single step with no explicit upfront plan — it decides the next action one at a time based on the current state. Plan-and-Execute instead commits to a structure before execution begins, which gives better global coherence for tasks with a clear multi-step structure (it won't wander) but is less adaptive moment-to-moment if the plan turns out to be based on wrong assumptions, unless the replanning step is triggered.

In practice, Plan-and-Execute tends to be more token-efficient for well-structured tasks (the plan is generated once, not re-derived at every step), while ReAct tends to be more robust for tasks where the right next step genuinely depends on what was just observed and can't be planned far in advance.

Q. What is Reflexion (self-reflection), and what problem does it solve that ReAct alone doesn't?

ADVANCED

Reflexion adds an explicit self-evaluation step after a task attempt: the agent (or a separate evaluator) assesses whether the attempt succeeded, and if not, generates a natural-language reflection on what went wrong — which is then stored and fed back into context on the next attempt, so the agent doesn't repeat the same mistake blindly.

This solves a real gap in plain ReAct: ReAct can get stuck retrying the same failed approach because it has no structured mechanism to step back and critique its own strategy — it just keeps reasoning forward from wherever it currently is. Reflexion's explicit critique-and-retry cycle gives the agent something closer to genuine iterative improvement across attempts, rather than just more of the same greedy step-by-step reasoning.

The cost is added latency and tokens (an extra evaluation pass, potentially multiple attempts), so it's best suited to tasks where getting it right matters more than getting it fast — coding tasks with a verifiable test suite are a canonical use case, since 'did it pass' gives a cheap, reliable signal for the reflection step to act on.

Q. What is Tree of Thoughts, and when is it worth the added complexity over a single reasoning chain?

ADVANCED

Tree of Thoughts generalizes chain-of-thought reasoning into a search over multiple possible reasoning paths: at each step, the model generates several candidate next-thoughts rather than committing to one, evaluates or scores those candidates, and explores the most promising branches — with the ability to backtrack from a branch that turns out to be unproductive, unlike a single linear chain.

It's worth the added complexity (more LLM calls, more latency, more cost) specifically for problems with a genuine branching structure where early mistakes are easy to make and costly to recover from within a single greedy chain — puzzle-solving, certain planning problems, or tasks with multiple plausible strategies where you can't tell which is right until several steps in.

For most everyday agentic tasks (tool-calling workflows, straightforward multi-step retrieval), a single well-structured reasoning chain (ReAct or Plan-and-Execute) is sufficient and far cheaper — Tree of Thoughts is a targeted tool for a specific class of hard, branching-search problems, not a general-purpose upgrade, and naming that distinction is what separates a candidate who's implemented it from one who's just read the paper title.

Q. How do you decide how much autonomy to grant a planning agent — full autonomous multi-step execution vs. plan-then-confirm?

ADVANCED

The key variable is reversibility and blast radius: for read-only or easily reversible actions (searching, drafting, summarizing), full autonomous execution across many steps is reasonable since a wrong turn costs little. For actions with real-world side effects that are hard or impossible to reverse (sending communications, spending money, modifying production systems), the plan should be surfaced for human confirmation before execution, even if the agent is fully capable of generating a correct plan on its own.

A common production pattern is staged autonomy: let the agent plan and execute freely within a sandboxed or low-stakes scope, but require explicit approval at defined checkpoints before crossing into higher-stakes actions — this gets most of the efficiency benefit of autonomy while keeping a human in the loop exactly where the cost of a mistake is highest, rather than treating 'autonomous' as an all-or-nothing switch.

Q. How does hierarchical planning (a high-level planner delegating to lower-level sub-agents or sub-tasks) help with very complex, long-horizon tasks?

ADVANCED

A flat, single-level agent trying to plan every low-level step of a long, complex task upfront tends to produce brittle, overly detailed plans that break as soon as reality diverges from assumptions — and it burns a huge amount of context trying to hold the entire plan in view at once.

Hierarchical planning splits this into levels of abstraction: a high-level planner decomposes the task into a small number of major milestones or sub-goals without specifying exactly how each will be achieved, and separate lower-level planners (or sub-agents) figure out the detailed steps for each sub-goal independently, only when that sub-goal becomes active.

This mirrors how humans plan complex projects — you don't plan every keystroke of a two-week project on day one — and it keeps each level's context focused and manageable, makes replanning cheaper (a failure in one sub-task only requires replanning that sub-task, not the whole project), and is a natural fit for combining with a supervisor/multi-agent architecture where the high-level planner is itself a distinct orchestrating agent.

03

Tool Use & Function Calling

Tools are how an agent touches the world. Interviewers probe this section hard because tool design mistakes are the single most common source of unreliable agents in production.

Q. What makes a good tool interface for an agent to call reliably?

FOUNDATIONAL

A clear, specific name and description that unambiguously conveys what the tool does and when to use it — vague descriptions are one of the most common causes of an agent picking the wrong tool or misusing the right one.

A tightly-scoped parameter schema with descriptive field names and explicit types/constraints (enums for fixed choices rather than free-text where possible) — the more the schema itself constrains valid input, the less the model has to 'get right' purely through reasoning.

Small, single-purpose tools generally outperform large, multi-purpose ones — a tool that does one thing well is easier for the model to select correctly and easier for you to test and secure independently, versus a single tool with a dozen optional parameters covering many different behaviors.

Q. How should tool error messages be designed so an agent can recover gracefully?

INTERMEDIATE

Error messages should be specific and actionable — 'Invalid date format, expected YYYY-MM-DD, got "next Tuesday"' gives the model something concrete to fix on retry, whereas a generic 'Error: request failed' gives it nothing to reason about, which is a very common cause of agents blindly retrying the exact same failed call.

Distinguish retryable errors (rate limits, transient timeouts) from non-retryable ones (invalid permissions, malformed input that will never succeed) in the error response itself where possible, so the agent (or the surrounding orchestration code) can decide whether retrying is even worth attempting rather than looping pointlessly.

Where feasible, suggest the fix directly in the error message rather than just describing the problem — 'Did you mean tool X instead?' or listing valid enum values inline — since this measurably improves an agent's ability to self-correct without burning additional reasoning steps rediscovering the constraint from scratch.

Q. How do you handle a situation where an agent has access to dozens of tools and starts picking the wrong one or getting confused?

ADVANCED

Beyond a certain number of tools (often cited around 10–20, though it's model- and task-dependent), tool selection accuracy degrades because the model has to discriminate between many similar-sounding options within a single context, which is a genuinely harder problem than choosing between a small, well-differentiated set.

Mitigations: group tools hierarchically (a top-level router decides a category, then a sub-agent or filtered tool list handles that category specifically) rather than exposing every tool flatly at once; dynamically filter the available tool list based on the current context/task stage instead of always exposing the full set; and consolidate near-duplicate or overlapping tools into one well-designed tool with parameters, since overlapping tools are a common direct cause of misselection.

This is also a strong argument for treating your tool set as a piece of API design that deserves the same rigor as any other interface — auditing for redundancy and ambiguity, not just adding a new tool every time a new capability is needed.

Q. What's the difference between parallel and sequential tool calling, and when does each make sense?

INTERMEDIATE

Sequential tool calling executes one tool call, waits for the result, reasons about it, then decides the next call — necessary when a later call genuinely depends on an earlier result (e.g., you need a user ID from one lookup before you can fetch that user's orders).

Parallel tool calling issues multiple independent tool calls at once when they don't depend on each other's results (e.g., checking weather in three different cities for a trip-planning request) — this reduces total latency significantly since the calls execute concurrently rather than waiting on each other unnecessarily.

Modern function-calling APIs increasingly support the model proposing multiple tool calls in a single turn specifically for this reason; a well-designed agent should recognize independence between requested actions and issue them together rather than defaulting to strictly sequential execution, which is a meaningful and easy-to-overlook latency optimization.

Q. How do you validate and sanitize tool arguments before executing them, even when they come from a 'trusted' model?

ADVANCED

Never trust model-generated arguments as inherently safe just because they matched the schema syntactically — schema validation confirms shape (correct types, required fields present), not semantic correctness or authorization (e.g., a syntactically valid file path might still point outside an allowed directory, or a valid-looking account ID might belong to a different user than the one making the request).

Apply the same input validation discipline you would to any untrusted user input: bounds-check numeric values, validate that referenced IDs/resources actually belong to the current authenticated context, sanitize any values that will be interpolated into a downstream query or command, and enforce authorization checks server-side rather than assuming the model 'wouldn't' request something out of scope.

The mental model worth stating explicitly in an interview: the LLM's output is user-adjacent untrusted input from the application's perspective, not privileged internal instruction — because the model's decision to call a tool with certain arguments can itself be the product of a prompt injection, not just an honest task-following decision.

Q. How would you design a tool that lets an agent execute arbitrary code (e.g., a code-interpreter tool), and what makes it safe?

ADVANCED

Execute in an isolated, ephemeral sandbox (a container or microVM) with no network access by default, strict CPU/memory/time limits, and a filesystem that's either fully ephemeral or scoped to a narrow, non-sensitive working directory — the sandbox boundary is the actual security control, not anything in the prompt asking the model to 'be careful.'

Tear down and recreate the sandbox between untrusted executions (or at least between different users' sessions) so state, including any malicious artifacts from a prior run, can't leak across executions or be exploited to affect a later one.

Cap resource usage aggressively (execution timeout, output size limits) since a code-execution tool is a natural target for both accidental runaway loops and deliberate resource-exhaustion attacks, and log all executed code for auditability — treat this tool with the security posture of running arbitrary untrusted user-submitted code, because that is functionally what it is, regardless of the fact that an LLM generated it.

04

Agent Memory Systems

Memory design is where a lot of agent projects quietly fall apart in production — context grows unbounded, relevant facts get lost, and costs balloon. This is a favorite deep-dive area for senior-level interviews.

Q. What's the difference between short-term (working) memory and long-term memory in an agent, and how is each typically implemented?

FOUNDATIONAL

Short-term memory is the current task's active context — the recent conversation, intermediate reasoning, and tool results within the ongoing session — held directly in the LLM's context window and typically discarded or summarized once the session ends.

Long-term memory persists across sessions: facts, preferences, or past decisions the agent should remember in future, unrelated interactions. It lives outside the context window entirely — usually in a database or vector store — and is selectively retrieved into the prompt only when relevant to the current task, rather than being permanently included.

The core engineering challenge is deciding what's worth promoting from short-term to long-term storage (not everything in a conversation is worth remembering forever) and how to retrieve the right long-term memories at the right moment without flooding the context with irrelevant history.

Q. What's the difference between episodic and semantic memory in an agent system?

INTERMEDIATE

Episodic memory stores specific past events or interactions — 'on this date, the user asked X and the agent did Y with this outcome' — preserving the context and sequence of a specific episode, useful for referencing 'what happened last time' or learning from a specific past success/failure.

Semantic memory stores generalized facts and knowledge extracted from experience, decoupled from any specific episode — 'this user prefers concise answers' or 'this codebase uses pytest, not unittest' — distilled, durable facts rather than a log of specific events.

Production agent memory systems often maintain both: episodic memory (often a simple log or vector-indexed history of past interactions) for specific recall, and a semantic memory layer that periodically extracts and consolidates durable facts from episodic history — similar to how a human might keep a diary (episodic) while also forming general beliefs and habits from repeated experience (semantic).

Q. How do you prevent an agent's context window from growing unboundedly across a long conversation or long-running task?

INTERMEDIATE

Summarization/compaction: periodically compress older turns into a running summary, keeping recent turns verbatim but replacing older history with a condensed representation — trading some fidelity for a bounded context size.

Selective retention: not all history is equally worth keeping — explicitly drop tool outputs or intermediate reasoning that's no longer relevant once a sub-task completes, rather than keeping every raw tool result in context indefinitely.

Externalize to long-term memory: move durable facts worth keeping into external storage (as discussed above) and retrieve them on demand, rather than trying to keep everything potentially useful in the active context window at all times.

The failure mode to flag explicitly in an interview: naive unbounded context growth doesn't just cost more money — it actively degrades quality, since models are demonstrably less reliable at using information buried in the middle of a very long context than information near the beginning or end ('lost in the middle').

Q. How would you design a memory retrieval system so an agent surfaces the right past memories at the right time, without flooding context with irrelevant ones?

ADVANCED

Standard approach: embed each stored memory and retrieve via similarity search against the current task/query, similar to RAG — but naive similarity-only retrieval for memory often surfaces things that are topically similar but not actually useful right now (e.g., an old preference that's since changed).

Improve relevance with additional signals beyond pure similarity: recency (recent memories often more relevant than old ones, especially for preferences that can change over time), frequency/importance (memories reinforced or referenced multiple times are more likely to be durable facts worth surfacing), and explicit memory metadata (tagging memories by type or topic to allow structured filtering, not just semantic search).

Cap the number of retrieved memories injected per turn and rank/re-rank them before inclusion, the same way you would for RAG document retrieval — memory is fundamentally a retrieval problem, and it benefits from the same rigor (avoiding noise, keeping the most relevant items, not just the most semantically similar ones) as any other RAG pipeline.

Q. What are the risks of an agent's long-term memory becoming stale or incorrect over time, and how do you mitigate them?

ADVANCED

Stale memory risk: a fact stored as true at one point (a user's role, a project's status, a preference) can become outdated, and if the agent keeps retrieving and acting on it without any invalidation mechanism, it will confidently act on wrong information — this is functionally the same failure mode as stale documents in a RAG index.

Contradiction risk: as new information comes in, it can conflict with previously stored memories rather than cleanly overwriting them, and without explicit conflict resolution, an agent might retrieve and use either the old or new fact inconsistently across different interactions.

Mitigations: version or timestamp memories and prefer more recent ones when conflicts arise, build an explicit update/invalidation path when new information supersedes an old memory rather than only ever appending, and periodically audit or expire memories that haven't been reinforced or referenced in a long time — treating long-term memory as a maintained store that needs the same hygiene as any other production data store, not an append-only log that's trusted forever.

Q. How does memory design differ between a single-user personal assistant agent and a multi-tenant enterprise agent?

ADVANCED

A single-user agent's memory can be relatively simple — one memory store per user, straightforward retrieval scoped to that one person's history, minimal risk of cross-contamination.

A multi-tenant enterprise agent needs strict memory isolation between tenants/users as a hard security boundary, not just a filtering convenience — a memory retrieval bug that surfaces one customer's data to another is a serious data-leakage incident, not just a relevance problem. This typically means memory storage needs the same access-control rigor as the underlying data systems (tenant-scoped indexes or hard filters enforced at the storage layer, not just prompt-level instructions to 'only use this user's data').

Enterprise contexts also often need shared organizational memory (facts relevant to a whole team, not one individual) layered alongside per-user memory, which adds a permission dimension — some memories should be visible to everyone on a team, others strictly private — that a personal-assistant use case usually doesn't have to reason about at all.

05

Multi-Agent Systems & Orchestration

Once a system involves more than one agent, coordination becomes the central engineering problem. This section tests whether you understand the real coordination patterns, not just that 'multiple agents talk to each other.'

Q. What is the supervisor (orchestrator-worker) pattern in multi-agent systems, and why is it the most common production pattern?

FOUNDATIONAL

A supervisor agent receives the overall task, decomposes it, and routes sub-tasks to specialized worker agents, then collects and synthesizes their results into a final response — workers don't communicate directly with each other, only with the supervisor.

This is the most common production pattern because it keeps coordination logic centralized and auditable — there's one place responsible for the overall plan and one place to debug when something goes wrong — versus fully decentralized peer-to-peer agent communication, which is powerful in principle but exponentially harder to reason about, test, and keep consistent as the number of agents grows.

It also maps cleanly onto the hierarchical planning pattern discussed earlier: the supervisor is effectively the high-level planner, and each worker is a specialized sub-agent handling one part of the decomposed task.

Q. What's the difference between a graph-based agent orchestration (like LangGraph) and a purely conversational multi-agent pattern (like AutoGen's group chat)?

INTERMEDIATE

Graph-based orchestration models the workflow as an explicit state machine — nodes represent agents or steps, edges represent allowed transitions, and the graph structure itself constrains what can happen next. This gives strong guarantees about control flow (you can't accidentally skip a required step) and makes the state easy to persist, inspect, and resume, at the cost of needing to design the graph structure upfront.

Conversational multi-agent patterns instead let agents participate in a shared conversation/group chat, with the sequence of who speaks next often decided dynamically (by a manager agent, a round-robin, or the conversation content itself) rather than a predefined graph — this is more flexible and closer to how humans collaborate, but harder to guarantee correctness or debug, since the actual sequence of events isn't fixed in advance and can vary between runs.

The practical tradeoff: graph-based orchestration suits well-defined, repeatable workflows where reliability matters most; conversational patterns suit more open-ended, exploratory collaboration where the right sequence of steps genuinely can't be known in advance.

Q. How do agents actually communicate and hand off work to each other — what does the message/state actually contain?

INTERMEDIATE

The naive approach — passing raw natural-language messages between agents, the way humans chat — is flexible but fragile, since a receiving agent has to correctly parse and interpret free-form text, which introduces the same miscommunication risk humans have, compounded by the fact that neither agent can ask real-time clarifying questions the way people naturally do.

A more robust pattern uses structured hand-offs: a typed schema (e.g., a JSON object with explicit fields like task, context, constraints, prior_results) rather than free-form prose, so the receiving agent has an unambiguous contract for what it's being asked to do and what information it has available — this significantly reduces the coordination failures that plague looser conversational hand-off patterns.

In practice, many production systems use a hybrid: a structured 'envelope' (task metadata, routing info) wrapping a natural-language payload (the actual instructions/context), giving both the reliability of structure and the flexibility of natural language where it's genuinely needed.

Q. What is a blackboard architecture in multi-agent systems, and when is it a better fit than a supervisor pattern?

ADVANCED

A blackboard architecture uses a shared, centralized state ('the blackboard') that all agents can read from and write to, rather than communicating through a central supervisor or direct peer-to-peer messages — agents independently monitor the blackboard and contribute when they have something relevant to add, without needing to know about each other directly.

This fits well when the set of relevant contributors isn't known in advance or the problem benefits from opportunistic, loosely-coordinated contribution — e.g., several specialist agents each contributing partial analysis to a shared evolving document, where the right contributor at any given moment depends on the current state of the blackboard rather than a fixed routing rule.

The tradeoff versus a supervisor pattern: a blackboard is more flexible and decentralized, but loses the supervisor's centralized control and easy auditability — race conditions on shared state (two agents writing conflicting updates) and unclear termination conditions (when is the blackboard 'done'?) are real engineering challenges that a supervisor pattern's centralized decision-making largely avoids by construction.

Q. How do you prevent (or detect and recover from) a deadlock or infinite hand-off loop between multiple agents?

ADVANCED

Prevention: enforce a global step/hand-off budget across the entire multi-agent session, not just per-agent — a supervisor should track total steps taken across all workers and hard-stop the whole system if it's exceeded, since per-agent limits alone don't prevent two agents from ping-ponging a task back and forth indefinitely.

Detection: log the sequence of hand-offs and check for repeating cycles (agent A hands to B, B hands back to A, with no progress on the actual task) — a simple cycle-detection check on the hand-off history catches this class of failure well before a hard step limit would.

Recovery: when a loop is detected, break out explicitly and escalate — either to a different resolution strategy, a fallback simpler approach, or a human — rather than either silently continuing to loop or silently failing with no explanation. As with single-agent loops, the underlying principle is the same: the system needs an explicit, code-enforced circuit breaker, because relying on the agents themselves to recognize and break their own loop is not a reliable control.

Q. What is a debate or voting pattern in multi-agent systems, and what problem is it actually trying to solve?

ADVANCED

In a debate/voting pattern, multiple agent instances (often the same underlying model, sometimes with different prompts or personas) independently attempt the same task or argue different positions, and either debate each other's answers or a final decision is made by majority vote or a separate judge model — conceptually similar to self-consistency sampling, but applied at the level of full independent agent runs rather than single reasoning chains.

The problem it addresses: a single agent's reasoning can be confidently wrong, and there's no internal signal that distinguishes a confident-correct answer from a confident-incorrect one from the outside. Independent attempts that converge on the same answer are a much stronger correctness signal than a single attempt, however articulate that attempt sounds.

The cost is substantial — running multiple full agent instances multiplies latency and cost roughly linearly with the number of participants — so this pattern is reserved for genuinely high-stakes decisions where the cost of being wrong clearly outweighs the added compute cost, not applied as a general reliability booster for routine tasks.

06

Agentic RAG & Retrieval Strategies

Agentic RAG is where retrieval and agency intersect — instead of a fixed retrieve-then-generate pipeline, the agent itself decides what, when, and how to retrieve. This is a frequent deep-dive for roles combining agents with knowledge-intensive tasks.

Q. What is agentic RAG, and how does it differ from standard RAG?

FOUNDATIONAL

Standard RAG follows a fixed pipeline: embed the query, retrieve top-k chunks, generate an answer — the same steps execute in the same order for every query, regardless of whether the first retrieval actually found what's needed.

Agentic RAG instead treats retrieval as one of several tools the agent can choose to invoke, potentially multiple times, with the agent deciding whether to retrieve at all, what to search for, whether the results are sufficient, and whether to reformulate the query and search again — the retrieval process itself becomes part of the adaptive agent loop rather than a fixed upstream step.

This matters most for queries where a single retrieval pass genuinely isn't enough — multi-hop questions, queries requiring iterative refinement, or situations where the agent needs to decide between multiple different knowledge sources — at the cost of added latency and complexity versus standard RAG's single, predictable retrieval step.

Q. What is corrective RAG (CRAG), and what specific failure mode does it address?

ADVANCED

Corrective RAG adds an explicit relevance-grading step after retrieval: before generation, a lightweight evaluator (often the LLM itself, or a smaller classifier) assesses whether the retrieved documents actually address the query. If they're judged irrelevant or insufficient, the system triggers a corrective action — reformulating the query and retrieving again, falling back to a web search, or explicitly acknowledging the knowledge gap — rather than proceeding to generate an answer from poor context.

This directly addresses a core standard-RAG failure mode: a fixed pipeline has no mechanism to notice that retrieval failed, so it generates an answer from irrelevant context regardless, which is a major source of ungrounded or hallucinated responses. Corrective RAG explicitly checks that assumption before committing to generation, trading some added latency for meaningfully better failure detection.

Q. How would you design an agent that must choose between multiple different retrieval sources (e.g., a vector DB, a SQL database, and a live web search)?

ADVANCED

Expose each retrieval source as a distinct, clearly-described tool (rather than a single generic 'search' tool) so the agent's tool-selection reasoning can match the query's actual information need to the right source — a question about current events routes to web search, a question about internal documents routes to the vector DB, a question needing precise aggregation routes to SQL.

Provide the agent with enough context to make this routing decision well: clear tool descriptions stating what kind of information each source contains and its freshness/limitations, and few-shot examples in the system prompt covering ambiguous routing cases if misrouting is observed in practice.

For cases where the right source genuinely isn't obvious from the query alone, consider a query-classification step before retrieval (either a dedicated lightweight classifier or the agent's own first reasoning step) rather than relying purely on tool descriptions to disambiguate — and allow the agent to query multiple sources and synthesize when a question genuinely spans more than one, rather than forcing a single-source decision.

Q. What is self-querying retrieval, and how does it improve on plain semantic search?

INTERMEDIATE

Self-querying retrieval uses the LLM to translate a natural-language query into a structured query — separating out explicit metadata filters (date ranges, categories, authors, numeric constraints) from the semantic portion of the search, rather than relying purely on embedding similarity to capture everything, including precise constraints embeddings handle poorly.

For example, 'find support tickets about billing issues from last month with priority high' gets decomposed into a semantic search for 'billing issues' plus explicit structured filters on date range and priority — rather than hoping the embedding similarity alone captures the precise date and priority constraints, which dense retrieval is generally weak at expressing accurately.

This is a strong example of agentic reasoning improving retrieval quality specifically because it recognizes that not every part of a query is best served by semantic similarity — some parts are precise, structured constraints that deserve precise, structured filtering, and letting the model make that decomposition is more robust than forcing everything through a single embedding.

Q. How do you prevent an agentic RAG system from retrieving excessively (looping on retrieval calls) or retrieving too little (giving up too early)?

ADVANCED

Excessive retrieval: cap the number of retrieval attempts per query with an explicit limit, and require the relevance-grading step (as in corrective RAG) to show the agent is making genuine progress — retrieving different, more targeted results on each attempt — rather than allowing unlimited re-querying, since an agent stuck reformulating a fundamentally unanswerable query can loop on retrieval just as easily as it can loop on any other tool.

Insufficient retrieval: this is subtler — an agent that retrieves once, finds partial results, and generates an answer from incomplete information looks superficially successful (it produced an answer) but may be silently wrong or incomplete. Mitigate by explicitly prompting the agent to assess retrieval sufficiency before generating (not just relevance, but coverage — does this actually answer the full question?), and by building this sufficiency check into your evaluation set so incomplete-but-plausible-sounding answers get caught before shipping, not just outright wrong ones.

07

Safety, Guardrails & Permissions for Agents

Agents that take real actions carry real risk. This is consistently the highest-scrutiny section in agentic AI interviews — a candidate who can't speak concretely to agent safety is a hard pass for most teams building production agents.

Q. What is the principle of least privilege as applied to agent tool access, and why does it matter more for agents than for typical application code?

FOUNDATIONAL

Least privilege means scoping each tool's actual permissions to the minimum required for its function — a tool that reads customer order status should have read-only access to the orders table, not broad database access, even if it would be more convenient to grant one broad credential to every tool.

It matters more for agents than typical application code because the agent's decision about which tool to call and with what arguments is made by a probabilistic model, not deterministic code — the same 'if this input, then this action' guarantees you'd get from hand-written logic don't hold. A prompt injection, a reasoning error, or an edge case the model handles poorly can all result in a tool being called in an unintended way, and least privilege is what limits the actual damage when — not if — that eventually happens.

Concretely: scope credentials per-tool rather than sharing one broad credential across an agent's whole toolset, and treat 'the model probably wouldn't do that' as never a sufficient safety argument on its own.

Q. Design a human-in-the-loop approval system for an agent that can take real-world actions. What actually needs approval, and how do you avoid approval fatigue?

ADVANCED

Tier actions by reversibility and blast radius rather than requiring approval uniformly: read-only or easily-reversible actions (searching, drafting a message that hasn't been sent) proceed autonomously; actions with real, hard-to-reverse consequences (sending a communication, making a purchase, deleting data, modifying a production system) require explicit human approval before execution.

Present approval requests with enough context to actually evaluate them quickly — the specific action, its arguments, and a concise summary of why the agent wants to take it — rather than forcing the approver to reconstruct context from a long agent trace, which is what actually causes approval fatigue and rubber-stamping.

Avoid over-triggering approval: if a low-risk action category is triggering approval requests constantly, that's a signal the tiering is miscalibrated, not a reason to just tolerate the friction — approval fatigue causes people to stop reading requests carefully and approve reflexively, which defeats the entire point of the control. Track approval-request volume and approval/rejection rates per action type as an operational metric, not just a one-time design decision.

Q. How does prompt injection risk differ for an agent with tool access compared to a standard chatbot, and how do you defend against it?

ADVANCED

For a standard chatbot, a successful prompt injection mostly risks producing an undesired or off-brand text response — bad, but generally low real-world consequence. For an agent with tool access, a successful injection can cause the agent to actually execute unauthorized real-world actions — send data somewhere it shouldn't, delete something, make an unauthorized purchase — which is a fundamentally higher-stakes failure mode, especially when the injection comes from untrusted content the agent processes as part of its task (a webpage, a document, a tool result), not just the direct user prompt.

Defenses specific to agents: treat all content the agent retrieves or receives from tools as untrusted data, never as instructions, and reinforce this distinction structurally (clear delimiters, explicit framing) rather than relying on the model to infer it; apply least-privilege tool scoping so even a successful injection has a bounded blast radius; require human confirmation for any action with real consequences, regardless of how the agent arrived at deciding to take it; and maintain a red-team regression suite specifically testing injection attempts embedded in tool results and retrieved documents, not just in direct user messages.

The honest framing for an interview: prompt injection for agents is not a solved problem, and the realistic posture is defense in depth — multiple imperfect layers, with the permission/approval boundary doing the most real work, not prompt wording.

Q. How would you sandbox an agent's actions during development and testing before granting it access to real production systems?

INTERMEDIATE

Use fully isolated staging environments with synthetic or anonymized data that mirror production tool interfaces exactly — the agent should interact with the same tool schemas and behaviors it will see in production, just pointed at safe, disposable resources, so testing genuinely reflects real usage rather than a simplified mock.

For tools with real-world side effects, build a 'dry run' or simulation mode that returns realistic responses without actually executing the action, allowing you to observe what the agent would have done across many test scenarios before it's ever granted the ability to actually do it.

Gate the transition from sandbox to production behind a genuine evaluation bar — a track record across a representative test suite, not just 'it worked in my manual testing' — and roll out production tool access incrementally (a limited action set, or a limited user population) rather than granting full production capability all at once on day one.

Q. What ongoing monitoring do you need once an agent with real-world tool access is live in production?

ADVANCED

Action-level audit logging: every tool call, its arguments, its result, and which task/session it was part of — this needs to be comprehensive enough to fully reconstruct what an agent did and why, after the fact, since agent behavior is inherently harder to predict than deterministic code and incidents will need to be investigated after they happen.

Anomaly detection on action patterns: unusual spikes in a specific tool's usage, unusual argument patterns, or actions outside normal operating parameters (e.g., a support agent suddenly requesting unusually broad data access) should trigger alerts, not just be logged passively.

Track approval/rejection rates for human-in-the-loop checkpoints over time — a rising rejection rate on a specific action type is a strong signal the agent's judgment for that action has degraded or the underlying task has shifted, and is worth investigating before it causes real harm rather than after.

08

Evaluating Agentic Systems

Evaluating an agent is a fundamentally harder problem than evaluating a single LLM response, because success depends on a whole trajectory of decisions, not one output. This is where senior candidates are most clearly differentiated.

Q. What's the difference between outcome-based and trajectory-based evaluation for agents?

FOUNDATIONAL

Outcome-based evaluation only checks the final result — did the agent achieve the intended goal, yes or no (or against some quality score) — regardless of how it got there.

Trajectory-based evaluation examines the full sequence of steps the agent took: which tools it called, in what order, with what arguments, and how it recovered from errors along the way — capturing whether the process was efficient, safe, and reasonable, not just whether it happened to land on the right answer.

Both matter and catch different failure classes: a correct outcome reached through a wasteful, fragile, or unsafe path (e.g., it got lucky, or it took an unauthorized action that happened not to cause visible harm this time) is a real risk that outcome-only evaluation completely misses — which is why rigorous agent evaluation checks the trajectory, not just whether the final answer looks right.

Q. Name some agent-specific benchmarks and what each is actually testing.

INTERMEDIATE

SWE-bench evaluates an agent's ability to resolve real GitHub issues in real codebases — it tests genuine multi-step software engineering capability (understanding a codebase, localizing a bug, writing a correct fix, passing existing tests) rather than an isolated coding puzzle.

WebArena and similar web-agent benchmarks test an agent's ability to complete realistic multi-step tasks by navigating and interacting with actual websites — testing tool-use reliability, planning, and error recovery in an environment that wasn't designed to be agent-friendly, which surfaces real-world brittleness that cleaner synthetic benchmarks miss.

AgentBench and similar broader suites test agent capability across multiple diverse environments (operating systems, databases, web shopping, games) specifically to measure generalization — whether an agent's planning and tool-use ability transfers across different domains, rather than being narrowly tuned to one task type.

The interview-relevant point: these benchmarks are useful for comparing base model capability, but they rarely reflect your specific production task closely enough to substitute for your own task-specific evaluation set — cite them to show awareness of the landscape, not as a replacement for building your own eval.

Q. How do you build a task-specific evaluation set for an agent you're building, from scratch?

ADVANCED

Start from real or realistic task scenarios spanning the difficulty range you actually expect in production — not just the easy happy-path cases, but ambiguous requests, cases requiring multiple tool calls, and known edge cases or previously-observed failure modes.

Define success criteria precisely per task before running the agent against it — for many agentic tasks, a specific, checkable outcome (a database row updated correctly, a specific file created with the right content) is a much more reliable signal than an LLM-judge's subjective assessment of a free-form final answer, so use checkable ground truth wherever the task allows it.

Where outcomes genuinely can't be checked automatically (open-ended research or writing tasks), use structured LLM-as-judge scoring against an explicit rubric, validated periodically against human judgment — and always evaluate a sample of the actual trajectory, not only the final output, to catch inefficient or risky paths that a pure outcome check would miss.

Treat this evaluation set as a living regression suite: every real production failure you diagnose should be added to it as a new test case, so the same class of failure is caught automatically before it can ship again.

Q. How would you use LLM-as-judge to evaluate an agent's full trajectory, and what are the specific pitfalls versus judging a single response?

ADVANCED

Provide the judge model with the full trajectory — every reasoning step, tool call, and observation, not just the final answer — along with an explicit rubric covering both outcome correctness and process quality (efficiency, appropriate tool use, safe handling of errors), since a judge shown only the final answer can't assess whether the path there was reasonable.

A specific pitfall beyond the general LLM-as-judge biases (position bias, verbosity bias): long trajectories can exceed what a judge model can meaningfully attend to in one pass, leading to it anchoring on the most recent or most salient steps rather than genuinely assessing the whole sequence — for very long trajectories, consider judging in structured segments or providing an explicit trajectory summary alongside the raw log rather than expecting the judge to reliably digest an extremely long raw transcript.

As with any LLM-as-judge setup, periodically validate the judge's trajectory assessments against a smaller human-labeled sample to confirm it's actually correlating with real quality judgments, not just producing plausible-sounding scores — this validation step is easy to skip and is exactly where judge reliability quietly drifts over time.

Q. How do you measure and think about cost-efficiency as an evaluation dimension for agents, not just task success?

INTERMEDIATE

Track steps-to-completion (how many tool calls / LLM turns it took to finish a task) alongside pure success rate — an agent that succeeds in 3 steps and one that succeeds in 15 steps are not equally good, even though both count as a 'success' under outcome-only evaluation, and the inefficient one costs more and has more surface area for something to go wrong along the way.

Track token cost and wall-clock latency per completed task as first-class evaluation metrics, not just an afterthought measured separately from quality — a common mistake is optimizing an agent for pure success rate in isolation and only discovering the cost/latency problem after it's already in production at scale.

When comparing two agent designs or prompt/model changes, always report the cost-quality tradeoff together (e.g., 'success rate improved 4%, but average cost per task increased 60%') rather than quality improvements in isolation — this is what lets a team make an informed decision about whether a change is actually worth shipping.

09

Agent Frameworks & Tooling

Interviewers use framework questions to check whether you understand the underlying patterns frameworks implement, not just which library you've used — you should be able to say what a framework buys you and what it costs.

Q. What is LangGraph's core abstraction, and what problem is it specifically designed to solve?

INTERMEDIATE

LangGraph models an agentic workflow as an explicit state graph: nodes represent processing steps (an LLM call, a tool call, a routing decision), edges represent allowed transitions between them (which can be conditional, based on the current state), and a shared state object flows through and is updated at each node.

This is specifically designed to solve the reliability and debuggability problems of loosely-structured agent loops — because the graph structure is explicit, you can visualize it, constrain which transitions are even possible, and persist/checkpoint the state at any node, enabling resumable execution after a failure or a human-approval pause. It essentially brings the rigor of a state machine to what would otherwise be an unconstrained loop of LLM calls deciding their own next step freely.

Q. How do CrewAI's role-based agents differ conceptually from LangGraph's graph-based approach?

INTERMEDIATE

CrewAI organizes multi-agent systems around role-based abstractions — you define agents with specific roles, goals, and backstories (e.g., 'a senior researcher' or 'a technical writer'), assign them tasks, and the framework handles orchestrating how they collaborate, generally modeled closer to how a human team might be structured and delegated to.

LangGraph instead exposes the underlying control-flow mechanics directly as a graph you design explicitly — it doesn't presuppose a 'team of role-based agents' framing at all, and can be used for a single agent's internal loop just as naturally as a multi-agent system.

The practical distinction: CrewAI's role-based framing can get a multi-agent collaboration pattern running faster for a common case with less custom orchestration code, at the cost of less fine-grained control over exact execution flow; LangGraph requires more upfront design of the graph but gives precise control over exactly what can happen and when, which tends to matter more as systems get complex or need strict reliability guarantees.

Q. What does 'checkpointing' mean in the context of agent frameworks, and why does it matter for long-running or production agents?

ADVANCED

Checkpointing means persisting an agent's full state (conversation history, current position in a graph/plan, intermediate results) to durable storage at defined points during execution, rather than only holding it in memory for the duration of a single process run.

It matters for long-running agents because without it, a process crash, a deployment, or a pause for human approval would lose all progress and force starting the task over from scratch — for a task that took many steps or significant cost to reach a certain point, that's a serious reliability and cost problem, not just an inconvenience.

Checkpointing also directly enables human-in-the-loop workflows: the agent's state can be checkpointed right before a high-stakes action, execution paused pending human approval (potentially for hours or days), and then resumed exactly where it left off once approved — without checkpointing, this pause/resume pattern isn't practically possible for anything beyond a trivially short wait.

Q. When would you choose to build a custom agent orchestration layer instead of adopting an existing framework, for a production system?

ADVANCED

When your control-flow requirements are simple and well-defined enough that a framework's general-purpose abstractions add overhead without adding real value — a straightforward single-agent tool-calling loop often doesn't need a full graph framework's machinery.

When you need behavior a framework doesn't cleanly support and would require fighting its abstractions to achieve — at that point, the framework is adding indirection without adding value, and a custom implementation is more maintainable long-term than a heavily-patched framework usage.

When deep debuggability and full transparency into exactly what's happening at each step is the top priority (common in early-stage or safety-critical systems) — a hand-rolled loop is easier to fully understand line-by-line than tracing through a framework's internal execution engine, which matters a lot when diagnosing a subtle production failure.

The strong interview answer: frameworks earn their complexity when you need their specific persistence/state-graph/multi-agent orchestration machinery for a genuinely complex workflow — treating framework adoption as a deliberate engineering tradeoff, not a default choice made because it's popular.

10

Deployment, Cost & Observability for Agents

An agent that works in a demo and an agent that's reliable and affordable in production are very different engineering problems. This section is where 'have you actually shipped this' gets tested.

Q. Why is cost control fundamentally harder for agents than for a single LLM call, and what levers do you have?

INTERMEDIATE

A single LLM call has a bounded, predictable cost — you know roughly what a request will cost before you make it. An agent's cost is a function of how many steps it takes to complete a task, and that number isn't fixed in advance — a task that normally takes 5 steps could take 20 if the agent gets stuck reasoning in circles, making agent cost inherently more variable and harder to forecast than single-call cost.

Levers: enforce hard step/iteration caps (bounds worst-case cost directly), route simpler sub-tasks to smaller/cheaper models within a multi-agent or hierarchical system rather than using your most expensive model for every step, cache repeated tool results and static context (prompt caching) across steps within a session, and monitor cost-per-completed-task as a first-class production metric so a regression (the agent starting to take more steps than it used to) gets caught quickly rather than silently inflating the bill.

Q. What does observability/tracing look like specifically for a multi-agent system, and how is it different from tracing a single-agent pipeline?

ADVANCED

For a single agent, a trace is a linear sequence of reasoning/action/observation steps — relatively straightforward to visualize and follow chronologically.

For a multi-agent system, you need to trace not just each individual agent's internal steps but also the hand-offs between agents — which agent initiated a hand-off, what state was passed, and how the receiving agent's subsequent actions relate back to that hand-off. This is closer to distributed tracing in traditional microservices (following a request across service boundaries) than to tracing a single linear process, and needs the same kind of correlation IDs tying related spans together across agent boundaries so a full task can be reconstructed end-to-end from the trace.

Without this, debugging a multi-agent failure becomes extremely difficult — you can see that the overall task failed, but not which agent's decision or which specific hand-off actually caused it, which is exactly the debugging problem multi-agent observability tooling exists to solve.

Q. How do you design retries and idempotency for an agent's tool calls, especially for tools with real-world side effects?

ADVANCED

For read-only or naturally idempotent tools (a lookup, a search), retrying on transient failure is straightforward and safe — repeating the exact same call produces the same effect with no additional risk.

For tools with real side effects (sending a payment, creating a record, sending a notification), a naive retry on a timeout is genuinely dangerous — the original call might have actually succeeded server-side even though the response was lost, and retrying blindly could cause the action to happen twice (double-charging, duplicate notifications).

The correct pattern is idempotency keys: the caller generates a unique key per logical action attempt, and the receiving system deduplicates on that key — a retried call with the same idempotency key is recognized as 'this already happened' and safely no-ops rather than re-executing. Any tool an agent can call with real side effects needs this supported at the tool/API level; it's not something the agent's prompt or retry logic alone can safely guarantee.

Q. How would you design the infrastructure to run thousands of concurrent long-running agent sessions reliably?

ADVANCED

Decouple agent execution from the request/response cycle for anything beyond a short task — use an async job/queue-based architecture where a task is submitted, executed by a pool of workers, and its state persisted (via the checkpointing discussed earlier) rather than holding a live connection open for the full duration of a potentially long task.

Scale the worker pool independently from the API layer, since agent execution is far more compute/token-intensive and variable in duration than typical request handling — treating agent workers similarly to how you'd scale any other async background-job worker fleet, with attention to per-session state isolation so one session's failure or resource spike doesn't affect others.

Plan explicitly for partial failure at scale: some fraction of concurrent sessions will hit transient tool failures, rate limits, or timeouts at any given moment purely due to volume — the system needs graceful degradation (queuing, backoff, partial-progress preservation via checkpointing) built in from the start, rather than treating failures as rare edge cases to handle later once you're already at scale.

11

Coding & Practical Exercises

Many agentic AI loops still include a live or take-home coding round. These exercises cover the control-flow building blocks that come up repeatedly — agent loops, supervisor routing, idempotent tool calls, and approval gates — with the reasoning an interviewer is actually listening for, not just a working solution.

CODING EXERCISE

Implement the core loop of a ReAct-style agent with tool calling and a hard step cap.

Interviewers want to see the reason → act → observe loop structured cleanly, an explicit max-iteration cap (a very common thing candidates forget), and a clear separation between 'the model decided to call a tool' and 'the application actually executes it' — the safety-relevant boundary discussed throughout this guide.

```
def run_agent(user_input: str, tools: dict, max_steps: int = 8) -> str:
    messages = [{"role": "user", "content": user_input}]

    for step in range(max_steps):
        response = llm_call(messages, tools=list(tools.keys()))

        if response.tool_call is None:
            return response.content  # model produced a final answer

        tool_name = response.tool_call.name
        tool_args = response.tool_call.arguments
        if tool_name not in tools:
            messages.append({"role": "tool", "content": f"Unknown tool: {tool_name}"})
            continue

        result = tools[tool_name](**tool_args)   # application executes, not the model
        messages.append({"role": "assistant", "tool_call": response.tool_call})
        messages.append({"role": "tool", "content": str(result)})

    return "Max steps reached without a final answer."
```

CODING EXERCISE

Implement a supervisor that routes tasks to worker agents and detects hand-off loops.

This checks whether you build in cycle detection (not just a step cap) and whether you track hand-off history explicitly rather than relying on the workers themselves to notice they're looping.

```
from collections import deque

def run_supervisor(task: str, workers: dict, max_handoffs: int = 12) -> str:
    history = deque(maxlen=4)
    current_worker, state = "triage", {"task": task}

    for _ in range(max_handoffs):
        result = workers[current_worker](state)
        history.append((current_worker, result.next_worker))

        if result.done:
            return result.output

        # cycle detection: same two-worker ping-pong repeating
        if len(history) == 4 and history[0] == history[2] and history[1] == history[3]:
            return "Escalating: detected a hand-off loop between workers."

        current_worker, state = result.next_worker, result.state

    return "Escalating: max hand-offs reached without completion."
```

CODING EXERCISE

Implement an idempotency wrapper for a tool with real-world side effects (e.g., issuing a refund).

Tests whether you understand idempotency keys as the actual fix for unsafe retries on side-effecting tools, not just 'add a retry loop.' A repeated call with the same key should no-op, not re-execute.

```
_processed_keys = {}  # in production: a durable store, not an in-memory dict

def idempotent_call(idempotency_key: str, action_fn, *args, **kwargs):
    if idempotency_key in _processed_keys:
        return _processed_keys[idempotency_key]  # already executed; return cached result

    result = action_fn(*args, **kwargs)
    _processed_keys[idempotency_key] = result
    return result

# usage:
# idempotent_call(f"refund:{ticket_id}:{attempt_id}", issue_refund, amount=42.00)
```

CODING EXERCISE

Implement context-window management: summarize older turns once history exceeds a token budget.

This checks whether you handle the boundary correctly (keep recent turns verbatim, summarize only what's being evicted) and whether you avoid re-summarizing on every single turn, which wastes tokens unnecessarily.

```
def manage_context(messages: list, max_tokens: int, keep_recent: int = 6) -> list:
    if estimate_tokens(messages) <= max_tokens:
        return messages

    to_summarize = messages[:-keep_recent]
    recent = messages[-keep_recent:]

    summary = llm_call([
        {"role": "system", "content": "Summarize this conversation history concisely, "
                                      "preserving key facts and decisions."},
        {"role": "user", "content": format_messages(to_summarize)},
    ]).content

    return [{"role": "system", "content": f"Earlier context summary: {summary}"}] + recent
```

CODING EXERCISE

Implement a tool-selection filter that limits the tools shown to an agent based on the current task stage.

This addresses the 'too many tools confuses the agent' problem directly — tests whether you dynamically scope the tool list instead of always exposing everything, and whether your filtering logic is explicit rather than hoping the model ignores irrelevant tools on its own.

```
TOOL_STAGES = {
    "research": ["web_search", "read_document", "self_query_kb"],
    "drafting": ["read_document", "save_draft"],
    "review":   ["save_draft", "request_approval"],
}

def get_available_tools(all_tools: dict, stage: str) -> dict:
    allowed_names = TOOL_STAGES.get(stage, list(all_tools.keys()))
    return {name: fn for name, fn in all_tools.items() if name in allowed_names}
```

CODING EXERCISE

Implement a human-in-the-loop approval gate for a destructive tool call.

Tests whether you make the approval boundary explicit and blocking (the action genuinely cannot execute without approval) rather than advisory, and whether you surface enough context for a human to actually evaluate the request quickly.

```
PENDING_APPROVALS = {}

def request_approval(action: str, args: dict, justification: str) -> str:
    request_id = generate_id()
    PENDING_APPROVALS[request_id] = {
        "action": action, "args": args,
        "justification": justification, "status": "pending",
    }
    notify_approver(request_id, action, args, justification)
    return request_id  # agent must wait; action does not execute yet

def execute_if_approved(request_id: str, execute_fn):
    request = PENDING_APPROVALS.get(request_id)
    if not request or request["status"] != "approved":
        raise PermissionError("Action not approved; cannot execute.")
    return execute_fn(**request["args"])
```

12

Scenario-Based Questions

These simulate the "here's a mess, walk me through your triage" questions that senior agentic AI interviews increasingly favor over textbook recall. Interviewers are scoring your diagnostic process — where you look first, how you isolate root cause, and what structural fix you'd ship — as much as the final answer.

SCENARIO

An autonomous agent with email and calendar tools sends a message to the wrong recipient because it misread ambiguous instructions. How do you respond, and what do you change?

- Immediate: assess actual impact (what was sent, to whom, any sensitive content involved) and take corrective action if possible (a follow-up correction, notifying the unintended recipient if appropriate) — treat this with the same seriousness as a human employee's mistake, not as a curiosity.
- Root-cause: was this an ambiguous-instruction problem (the agent had no way to know it was wrong), a tool-design problem (recipient selection was too error-prone), or a missing-guardrail problem (this action should have required confirmation and didn't)? The fix differs substantially depending on which it was.
- Structural fix: for any communication tool with external-facing consequences, add a human-confirmation step before sending, especially when recipient selection involves any ambiguity — this is exactly the class of action the least-privilege/human-in-the-loop framework is designed to catch.
- Add this exact scenario (and variations) to your evaluation and red-team suite so the same ambiguity pattern is caught in testing before it can recur in production.

SCENARIO

A multi-agent research system produces a final report that looks polished and well-cited, but on inspection, two of the 'facts' were fabricated by one of the sub-agents and never caught by the reviewer agent. Diagnose and fix.

- This is a trajectory failure hiding behind a good-looking outcome — exactly the scenario outcome-only evaluation misses. Pull the full trajectory: which sub-agent introduced the fabricated claims, and why did the reviewer agent's check fail to catch them?
- Common root cause: the reviewer agent was likely evaluating surface plausibility and internal consistency rather than actually verifying each claim against a source — a reviewer prompt that says 'check this for accuracy' without requiring explicit citation-to-source verification for each claim will reliably miss fabrications that merely sound coherent.
- Fix: require the reviewer (or the original agent) to tie every factual claim to a specific, checkable source at generation time, not just plausibility-review after the fact, and have the reviewer's job be explicitly to verify that mapping — 'does this citation actually support this claim' — rather than a general quality pass.
- Add explicit fabrication-detection cases to the evaluation suite, and consider adding this to the same class of faithfulness-checking used in RAG systems, since a multi-agent research pipeline has fundamentally the same grounding problem as RAG generation.

SCENARIO

A coding agent with sandboxed execution access starts consuming an unusual amount of compute in its sandbox, well beyond what similar past tasks required. What's your triage process?

- Check for a runaway loop first — an agent stuck retrying a failing test or repeatedly attempting the same fix without success is the most common cause of unusual compute consumption, and should be caught by a step/time cap if one is properly enforced.
- Check whether the sandbox's resource limits (CPU, memory, execution time) are actually being enforced correctly — a missing or misconfigured limit is a real possibility, not just an agent behavior problem, and is worth verifying directly rather than assuming the agent is at fault.
- Inspect the actual code being executed in the sandbox for the affected sessions — this could also indicate the agent generated genuinely resource-intensive code (an inefficient algorithm, an accidental infinite loop in generated code itself, not just in the agent's outer control loop) rather than an agent-orchestration-level problem.
- Once diagnosed, tighten whichever layer failed — outer step caps if it's an orchestration loop, sandbox resource limits if those weren't enforced, or code-review/static-analysis gates before execution if the generated code itself is the problem.

SCENARIO

Two agents in a supervisor-worker system keep handing the same sub-task back and forth without making progress, and the session eventually times out. Walk through your fix.

- This is a classic multi-agent hand-off loop — first confirm it via the trace: is the same task genuinely bouncing between the same two agents with no state change, or is there subtle progress that just isn't visible without a close read?
- If it's a genuine loop: check whether the supervisor's routing logic has a way to detect 'this hasn't changed since I last routed it here' — if not, that's the structural gap, since a supervisor that routes purely based on current state with no memory of prior routing attempts can't tell the difference between forward progress and a repeat.
- Fix: add explicit cycle detection to the supervisor (track recent hand-off history, not just current state) and a global step budget across the whole multi-agent session as a hard backstop, not just a per-agent one.
- Consider whether the underlying task decomposition was the actual root cause — if two agents keep needing to hand a task back and forth, that's sometimes a sign the task wasn't cleanly separable into independent sub-tasks in the first place, and the fix might be redesigning the decomposition, not just adding loop detection on top of a flawed split.

SCENARIO

An agent with tool access is discovered to have been manipulated via a webpage it visited during a research task — the page contained hidden text instructing it to exfiltrate conversation data to an external URL. The attempt was blocked by a tool permission check. What do you do next?

- Treat the permission check that blocked it as validation that least-privilege scoping is working as designed — and treat the fact that the injection reached that point at all as confirmation that prompt-level defenses alone did not and will not reliably prevent this class of attack.
- Audit whether the specific tool involved (whatever allowed the exfiltration attempt to even be considered) has appropriately scoped permissions — should this agent have had that capability available at all during a research task, or was the tool over-provisioned for what the task actually needed?
- Add this exact injection pattern (hidden instructions embedded in retrieved web content) to your red-team regression suite, and audit other tools with similar external-content exposure (documents, emails, any content not directly authored by the user) for the same risk class.
- Consider adding output-side monitoring specifically for anomalous data-exfiltration-shaped tool calls (unexpected outbound requests, unusual destinations) as a defense-in-depth layer beyond input-side permission checks alone.

SCENARIO

Your agent's success rate on your evaluation suite is excellent (95%+), but real users report frequent frustration with it in production. What's likely going wrong?

- This is a strong signal of an evaluation-set/production-traffic mismatch — the eval suite likely doesn't represent the actual distribution or difficulty of real user requests, the same failure pattern that shows up when a fine-tuned model scores well offline but underperforms live.
- Check whether your eval set's success criteria capture what users actually care about — a technically 'successful' outcome by your metric (task completed) might still involve excessive back-and-forth, an unnecessarily roundabout path, or a technically-correct-but-unhelpful final answer that a binary success/fail metric doesn't penalize.
- Pull a sample of real production trajectories (not just outcomes) and review them directly — trajectory-level review often surfaces exactly the kind of inefficiency or awkwardness that outcome-only evaluation is structurally blind to.
- Use what you find to both fix the immediate gaps and directly expand the evaluation set with real production examples, so the eval suite starts actually representing the traffic and quality bar users experience, closing the gap for future iterations.

SCENARIO

An agent occasionally calls a destructive tool (delete/cancel) with a plausible-sounding justification, but on review, the justification was based on a misread of ambiguous user intent. How do you redesign to prevent this?

- This is precisely the class of action that should never execute autonomously regardless of how confident or well-reasoned the agent's justification sounds — a plausible-sounding justification is not the same as a correct one, and destructive/irreversible actions need a hard confirmation gate independent of the agent's own confidence.
- Redesign the tool interface itself: for destructive actions, consider requiring the agent to explicitly restate what it's about to do and why in a structured confirmation payload shown directly to the human approver, rather than the approver having to infer intent from a long agent trace.
- Where user intent is genuinely ambiguous, the correct agent behavior is to ask a clarifying question before acting, not to proceed on its best guess — if the agent isn't currently prompted/designed to recognize and flag ambiguity for destructive actions specifically, that's the structural gap to close.
- Add ambiguous-intent test cases specifically targeting destructive tools to your evaluation suite, since this is a high-consequence failure mode worth deliberately over-testing relative to its raw frequency.

SCENARIO

A long-running agent task that was checkpointed and paused for human approval resumes several hours later, but its actions after resuming are inconsistent with the state it had before pausing. Diagnose the bug class.

- This points to a state-serialization or staleness bug in the checkpointing system — either the checkpoint didn't capture the full relevant state (something needed for correct continuation wasn't persisted), or external context referenced by the checkpointed state changed during the pause window and the resumed agent is now reasoning from stale assumptions.
- Check specifically whether any tool results or retrieved data embedded in the checkpointed state could have gone stale during the pause — data that was fresh when checkpointed but is now outdated hours later is a very plausible cause of behavior that looks 'inconsistent' upon resume.
- Fix: for any checkpoint that may be resumed after a meaningful time gap, consider explicitly re-validating or re-fetching time-sensitive data on resume rather than trusting the checkpointed snapshot as still accurate — the appropriate staleness tolerance depends entirely on the specific data and task.
- Add a resume-after-delay test specifically to your evaluation suite (checkpoint, simulate a time gap with changed underlying state, resume, and verify consistent behavior) since this failure mode won't be caught by tests that only checkpoint and resume immediately.

SCENARIO

Leadership wants to grant an existing well-tested internal agent significantly more autonomy (removing several human-approval checkpoints) to speed up throughput. How do you evaluate whether that's safe?

- Don't evaluate this as a binary — break down exactly which checkpoints are being proposed for removal and assess each independently against the reversibility/blast-radius framework: some may be safe to remove (low-stakes, easily reversible), others may not be, and a blanket 'remove several' proposal deserves to be examined checkpoint by checkpoint.
- Look at the historical approval/rejection data for each checkpoint being considered — a checkpoint with a near-zero historical rejection rate over a large volume of real cases is a much stronger case for removal than one with a track record of catching real problems, even if the latter feels like it's 'usually fine.'
- Propose a staged rollback-capable rollout rather than an irreversible full removal: pilot the reduced-approval version on a limited scope or traffic percentage first, with the same production monitoring discussed elsewhere in this guide (anomaly detection, audit logging) specifically watching for problems the removed checkpoint would have caught.
- Be direct with leadership about the actual tradeoff being made — throughput gains are real and quantifiable, but so is the increased blast radius of an undetected failure, and that tradeoff should be an explicit, informed decision, not something that gets glossed over in pursuit of a speed metric.

SCENARIO

Your team's agent framework was recently upgraded to a new major version, and several previously-passing evaluation cases started failing intermittently. How do you approach this regression?

- Before assuming it's a genuine capability regression, check whether the framework upgrade changed any default behaviors relevant to your agent's control flow — default retry counts, timeout values, or state-handling changes between major versions are a common, underappreciated source of 'mysterious' behavior changes after an upgrade.
- Given the failures are intermittent rather than consistent, suspect a timing- or concurrency-related change specifically — race conditions in state handling, altered async execution behavior, or changed default timeouts are the most likely category of cause for intermittent (vs. deterministic) regressions.
- Bisect if possible: test against the framework version immediately before the upgrade with everything else held constant to confirm the framework change is actually the cause, rather than a coincidental unrelated change landing around the same time.
- Once confirmed, decide between pinning to the last known-good version while you investigate further, or fixing forward with the new version — but either way, add the specific failing cases to your regression suite permanently, and treat framework upgrades going forward as changes that require a full run of your evaluation suite before merging, not just a changelog read-through.

13

System Design Case Studies

Senior agentic AI interviews almost always include one open-ended design question. These three walkthroughs model the structure and depth expected: architecture, tool/permission design, safety guardrails, and evaluation strategy — not just a framework name.

Design an autonomous customer-support agent with access to a CRM, a knowledge base, and the ability to issue refunds

- Architecture: a supervisor agent classifies intent and routes to either a knowledge-retrieval flow (agentic RAG over the knowledge base, with corrective retrieval if initial results are insufficient) or an action flow (CRM lookups, refund issuance) — kept as distinct paths so read-only knowledge questions never touch the higher-risk action tools unnecessarily.
- Tool design: CRM lookup tools are read-only and freely callable; refund issuance is scoped with a hard dollar-amount ceiling for autonomous execution and requires human approval above that threshold — least privilege applied concretely, not just as a principle.
- Memory: short-term conversation context plus a per-customer episodic memory of past interactions (previous tickets, prior resolutions) retrieved at session start, with strict tenant/customer isolation enforced at the storage layer, not just prompt-level instruction.
- Guardrails: PII handling review before any data leaves the system, refund-amount confirmation UI for anything above the autonomous threshold, and a red-team suite specifically covering social-engineering attempts to get unauthorized refunds approved.
- Evaluation: task success rate on a labeled ticket-resolution eval set, refund-accuracy audit (were refunds issued correctly and only when warranted), and trajectory review sampling to catch inefficient or borderline-inappropriate paths that a pure resolution-rate metric would miss.

Design a coding agent (like an autonomous PR-generating assistant) with sandboxed code execution

- Architecture: Plan-and-Execute or ReAct loop with access to file-read/write tools scoped to a specific repo checkout, a sandboxed code-execution tool for running tests, and a version-control tool for creating branches/commits — each tool scoped narrowly to its specific function rather than one broad 'run arbitrary shell command' tool.
- Sandbox: fully isolated, ephemeral execution environment per session with no network access, strict resource/time limits, and no access to secrets or credentials beyond what's needed for the specific repo — torn down and rebuilt between sessions.
- Safety: generated code changes are never auto-merged — every change surfaces as a draft PR for human review; the agent is never granted direct push access to protected branches regardless of how confident its own self-assessment is.
- Evaluation: SWE-bench-style task success rate as a baseline capability signal, plus a task-specific eval set built from your own repo's real historical issues, tracking both resolution rate and whether generated tests actually exercise the fix (not just whether existing tests still pass).
- Observability: full trace of every file read/write, every test run and its result, and every reasoning step — since debugging a coding agent's mistake fundamentally requires reconstructing exactly what it saw and did, not just the final diff.

Design a multi-agent system that produces a structured research report by gathering information from multiple sources

- Architecture: supervisor agent decomposes the research question into sub-topics, dispatches specialist researcher agents (each with web search and/or internal-document retrieval tools) per sub-topic in parallel where sub-topics are independent, then a synthesis/writer agent combines findings into the final report.
- Grounding: every factual claim in the final report must be traceable to a specific source gathered by a researcher agent — enforced by requiring researcher agents to output claims with explicit source citations, and having a dedicated reviewer agent verify each citation actually supports its claim before the report is finalized (directly addressing the fabrication-under-review failure mode).
- Coordination: structured hand-off schemas between researcher agents and the synthesis agent (explicit fields: claim, source, confidence) rather than free-form prose hand-offs, to keep the synthesis step reliable and auditable.
- Guardrails: cap total research depth/steps per sub-topic to bound cost, and require a minimum source-diversity check (not relying on a single source for a claim that materially affects the report's conclusions) before finalizing.
- Evaluation: trajectory-based review of a sample of full research runs (not just final reports), a dedicated fabrication/citation-accuracy check as a hard evaluation gate, and cost-per-report tracking given the multi-agent parallel research pattern's cost sensitivity.

14

Project & Portfolio Questions

These questions ask you to narrate your own agentic work. The difference between a strong and weak answer is almost never the project itself — it's whether you can name specific architecture decisions, tradeoffs, failures, and metrics instead of describing a generic agent.

PROJECT & PORTFOLIO

Walk me through an agentic system you built, end to end.

Structure around the architecture decisions, not just the outcome: single- vs. multi-agent and why, which planning/reasoning strategy you used and why it fit the task, the specific tools you built and how you scoped their permissions, and how you evaluated it before and after shipping. Name at least one concrete failure mode you hit (a loop, a tool misuse, a cost spike) and the structural fix — a project description with no failures mentioned reads as untested, not as a smooth success.

PROJECT & PORTFOLIO

Describe a time an agent you built took an unintended or incorrect action. What happened, and what did you change?

This is testing your safety instincts as much as your debugging process. Strong answers cover: what actually happened and its real-world impact (however small), the root cause (ambiguous instructions, a tool-design gap, a missing guardrail), the immediate fix, and — critically — the structural change that prevents the same class of failure from recurring, not just a patch for that one instance.

PROJECT & PORTFOLIO

Tell me about a multi-agent system you designed. Why did you choose that architecture over a single agent, and what coordination challenges came up?

Be honest about the tradeoff you made — multi-agent systems add real coordination overhead, so a good answer explains what specific benefit justified that cost for this task. Name a concrete coordination failure you hit (a loop, a miscommunicated hand-off, inconsistent shared state) and how you fixed it. If you've since concluded a single agent would have been simpler for a similar future task, saying so is a stronger signal of judgment than defending the multi-agent choice unconditionally.

PROJECT & PORTFOLIO

Describe how you evaluated an agent before putting it in production. What did your evaluation set look like, and what did it catch?

Cover both outcome and trajectory evaluation, how you built the evaluation set (real vs. synthetic scenarios, how you covered edge cases), and ideally a specific example of a regression or issue your evaluation suite actually caught before it reached users — that's a much stronger signal than a description of your evaluation philosophy in the abstract.

PROJECT & PORTFOLIO

Tell me about the guardrails or permission system you designed for an agent with real-world tool access.

Walk through your least-privilege scoping decisions, what specifically required human approval and why, and how you avoided approval fatigue if applicable. If you have a concrete story of a guardrail catching (or, if it happened, failing to catch) a problem, that's the strongest possible answer — it shows the safety design was actually load-bearing, not theoretical.

PROJECT & PORTFOLIO

Describe the memory system you designed for a long-running or multi-session agent.

Cover what you chose to persist as short-term vs. long-term memory and why, how retrieval worked (pure similarity vs. additional signals like recency), and how you handled context growth over long sessions. Mentioning a specific staleness, contradiction, or context-overflow problem you hit and fixed is much stronger than a purely descriptive answer about the memory architecture.

15

Behavioral & Judgment Questions

Agentic AI roles are judged as much on judgment and communication as on technical depth — especially around autonomy and safety tradeoffs. These questions probe how you think about failure, risk, and cross-functional work — prepare a real, specific story for each, not a generic answer.

BEHAVIORAL

How do you decide whether a problem actually needs an agent, versus a simpler single-call LLM feature or a traditional deterministic pipeline?

Strong answers describe a real decision framework: does the task genuinely require an unpredictable, variable-length sequence of steps decided at runtime, or can the steps be predetermined? Agents add real cost, latency, and failure-mode complexity, so the answer should show a bias toward the simplest solution that solves the problem, reaching for agentic complexity only when the task's structure actually demands it — not because agents are the current trend.

BEHAVIORAL

Tell me about a time you had to push back on giving an agent more autonomy than you thought was safe.

This tests both technical judgment and the ability to hold a position under business pressure. A strong answer names the specific risk you identified (concretely, not vaguely), how you communicated the tradeoff to stakeholders who wanted the autonomy, and how it was actually resolved — whether you held the line, found a staged compromise, or were overruled and what happened as a result. Avoid a story where you were simply right with no real tension.

BEHAVIORAL

How do you approach debugging an agent failure when the trajectory looks reasonable at every individual step, but the overall outcome was wrong?

Strong answers describe a systematic process: reviewing the full trajectory for compounding small errors rather than looking for one obvious mistake, checking whether the task decomposition or initial plan was flawed even if each execution step was locally reasonable, and distinguishing a tool/data problem from a reasoning problem. This question is checking for structured debugging instinct specific to agents, where failures are often systemic rather than a single identifiable bug.

BEHAVIORAL

Describe how you communicate the risk and reliability tradeoffs of an agentic system to non-technical stakeholders who want to ship fast.

Look for concrete translation skills: framing technical risk (hallucination, tool misuse, loop failures) in terms stakeholders actually care about (cost, customer trust, legal/compliance exposure), and proposing staged rollout or reduced-scope alternatives rather than just saying no. A good answer balances genuine safety concern with a real path to shipping, rather than either blocking everything or caving to pressure to skip safety work.

BEHAVIORAL

How do you stay current with agentic AI specifically, given how fast the frameworks and techniques are changing?

Look for specificity: naming actual practices (building small proof-of-concept agents with new techniques rather than only reading about them, following specific research groups or practitioners, critically evaluating new frameworks against real production experience) rather than a generic answer. Bonus signal if the candidate can articulate an opinion on a recent development (a new framework, a new benchmark) rather than just being aware it exists.

BEHAVIORAL

Tell me about a disagreement you had with a teammate over agent architecture (e.g., single vs. multi-agent, framework choice, autonomy level) and how it was resolved.

This is checking both technical reasoning and collaboration skill. A strong answer names the actual technical crux of the disagreement, how each side's reasoning was evaluated (ideally against some concrete evidence — a prototype, a benchmark, a cost estimate — rather than pure opinion), and the outcome, including what you learned from the process even if your original position didn't win.

16

General Behavioral & HR Questions

Standard cross-role behavioral and HR-round questions that show up in most agentic AI loops alongside the technical rounds. These aren't agent-specific, but skipping prep for them is one of the most common reasons a technically strong candidate stumbles late in the process.

HR & GENERAL BEHAVIORAL

Tell me about a conflict you had with a coworker or manager and how you resolved it.

Pick a real, specific disagreement (technical direction, priorities, or approach) rather than a vague or interpersonal-drama story. Structure it as: what the disagreement actually was, how you approached the other person (did you seek to understand their reasoning before pushing your own), what the resolution was, and what you'd do differently now. Interviewers are listening for maturity and directness, not conflict avoidance or a story where you were simply right and the other person was wrong.

HR & GENERAL BEHAVIORAL

What's your biggest weakness, and what are you actively doing about it?

Avoid the classic non-answer disguised as a strength ('I work too hard'). Name a real, specific, professionally relevant weakness, then focus most of the answer on the concrete, ongoing action you're taking to address it — a specific habit, process, or feedback loop you've put in place — rather than a vague intention to 'get better.' A credible, still-in-progress growth story reads far better than a weakness that's suspiciously already fully solved.

HR & GENERAL BEHAVIORAL

Why do you want to work here, and why this role specifically?

Generic answers ('great culture,' 'exciting mission') are instantly forgettable. A strong answer connects something specific about the company's actual product, technical challenges, or stage to something specific about your background or what you want to grow into next — showing you've done real homework, not just applied broadly. If you can name a specific technical problem this team is likely facing and why it interests you, that's far stronger than generic enthusiasm.

HR & GENERAL BEHAVIORAL

Tell me about a time you failed. What did you learn from it?

Choose a genuine failure with real stakes and a clear consequence — not a humble-brag disguised as a failure. Walk through what went wrong, your role in it (own your part specifically, don't just describe circumstances), and most importantly, a concrete change in how you work now as a direct result. The learning should be specific and demonstrable, not a generic platitude like 'I learned to communicate better.'

HR & GENERAL BEHAVIORAL

How do you prioritize when everything feels urgent?

Strong answers describe an actual framework or heuristic you use — e.g., distinguishing genuinely urgent-and-important work from merely loud requests, assessing blast radius/impact if something slips, and communicating tradeoffs explicitly to stakeholders rather than silently trying to do everything. A concrete example where you had to explicitly deprioritize something (and how you communicated that) is much stronger than an abstract description of a prioritization philosophy.

HR & GENERAL BEHAVIORAL

Where do you see your career going over the next few years?

The interviewer is checking for realistic self-awareness and alignment with what this role can actually offer — not a rehearsed, overly specific title-and-timeline plan. A strong answer names the kind of technical depth or scope you want to grow into, connects it credibly to what this role/company can realistically provide, and avoids either extreme: no vision at all, or a trajectory that obviously outgrows this role within a year.

Quick-Reference Glossary

A last-minute skim sheet — the patterns and terms most likely to come up as rapid-fire follow-ups.

Agent loop	Observe -> Reason -> Act -> repeat, with an explicit stopping condition
ReAct	Interleaves reasoning traces with actions; greedy, sequential, no backtracking
Plan-and-Execute	Generate full plan upfront, then execute step by step, replanning if needed
Reflexion	Adds self-evaluation + reflection after failed attempts to avoid repeating mistakes
Tree of Thoughts	Searches multiple reasoning branches with backtracking; costly, for hard branching problems
Supervisor pattern	Central orchestrator routes sub-tasks to specialist worker agents
Blackboard architecture	Shared state all agents read/write; decentralized, opportunistic contribution
Agentic RAG	Agent decides whether/what/when to retrieve, as a tool call within its loop
Corrective RAG (CRAG)	Grades retrieval relevance before generation; triggers requery/fallback if insufficient
Least privilege (agents)	Each tool scoped to minimum permissions needed; bounds blast radius of any failure
Idempotency key	Unique key per action attempt so retries of side-effecting tools don't duplicate the action
Checkpointing	Persisting agent state durably so long-running tasks can pause/resume without losing progress
Trajectory evaluation	Judging the full sequence of agent steps, not just the final outcome
Episodic vs semantic memory	Episodic = specific past events. Semantic = generalized, durable facts
Deadlock detection	Cycle-detection on agent hand-off history plus a global step budget as backstop

Before You Walk In

TECHNICAL PREP

- Be ready to draw the agent loop (observe → reason → act → repeat) from memory and narrate a concrete example out loud.
- Practice the single-agent-vs-multi-agent decision framework until it's automatic — it comes up in nearly every interview, and "it depends, here's why" is the right instinct.
- Have one real agentic project you can describe end-to-end: the architecture, your tool/permission design, what broke, how you evaluated it, and what you'd change.
- Know your position on autonomy and safety concretely — least privilege, human-in-the-loop tiering, and a real story where a guardrail mattered read far stronger than abstract safety talk.

IN THE INTERVIEW

- When asked "how would you build this agent," always state your autonomy/reversibility tradeoffs out loud — interviewers weight that judgment as much as the architecture itself.
- If you don't know something, say so plainly, then reason toward an answer from first principles — this is usually scored higher than confident guessing.
- For system design questions, structure your answer: architecture → tool/permission design → guardrails → evaluation → failure modes.
- Ask clarifying questions before diving into a design — expected autonomy level, blast radius of actions, and latency needs change the right answer completely.